

```

import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("bank.csv", sep=";")

df.head()
print(df.info())
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for column in df.columns:
    if df[column].dtype == "object":
        df[column] = le.fit_transform(df[column])

df.head()
X = df.drop("y", axis=1)

y = df["y"]

print(X.head())
print(y.head())
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.shape)
print(X_test.shape)
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

print("Model Trained Successfully")
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for column in df.columns:
    df[column] = le.fit_transform(df[column].astype(str))

print(df.dtypes)
X = df.drop("y", axis=1)

y = df["y"]
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

```

```

df = pd.read_csv("bank.csv", sep=";")
le = LabelEncoder()

for column in df.columns:
    if df[column].dtype == "object":
        df[column] = le.fit_transform(df[column])

X = df.drop("y", axis=1)
y = df["y"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

print("Model Trained Successfully")
df.dtypes
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20,10))

plot_tree(
    model,
    filled=True,
    feature_names=X.columns,
    class_names=["No", "Yes"]
)

plt.title("Decision Tree Classifier")

plt.show()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4521 entries, 0 to 4520
Data columns (total 17 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         4521 non-null   int64

```

1	job	4521	non-null	object
2	marital	4521	non-null	object
3	education	4521	non-null	object
4	default	4521	non-null	object
5	balance	4521	non-null	int64
6	housing	4521	non-null	object
7	loan	4521	non-null	object
8	contact	4521	non-null	object
9	day	4521	non-null	int64
10	month	4521	non-null	object
11	duration	4521	non-null	int64
12	campaign	4521	non-null	int64
13	pdays	4521	non-null	int64
14	previous	4521	non-null	int64
15	poutcome	4521	non-null	object
16	y	4521	non-null	object

dtypes: int64(7), object(10)

memory usage: 600.6+ KB

None

	age	job	marital	education	default	balance	housing	loan
contact \								
0	30	10	1	0	0	1787	0	0
0								
1	33	7	1	1	0	4789	1	1
0								
2	35	4	2	2	0	1350	1	0
0								
3	30	4	1	2	0	1476	1	1
2								
4	59	1	1	1	0	0	1	0
2								

	day	month	duration	campaign	pdays	previous	poutcome
0	19	10	79	1	-1	0	3
1	11	8	220	1	339	4	0
2	16	0	185	1	330	1	0
3	3	6	199	4	-1	0	3
4	5	8	226	1	-1	0	3
0	0						
1	0						
2	0						
3	0						
4	0						

Name: y, dtype: int64

(3616, 16)

(905, 16)

Model Trained Successfully

age int64

job int64

```
marital      int64
education    int64
default      int64
balance      int64
housing      int64
loan         int64
contact      int64
day          int64
month        int64
duration     int64
campaign     int64
pdays       int64
previous     int64
poutcome     int64
y            int64
dtype: object
Model Trained Successfully
Accuracy: 0.8751381215469614
```

